

Author: Dr Mike Lakoju

 Screenshot%202021-01-25%20at%203.56.38%20pm.png

- * If X is high, the value is approximately 1
- * if X is small, the value is approximately 0

Unit 7, Exercise 3: Multi-layer perceptron - Sigmoid function

The aim of this activity is to understand how a multi-layer perceptron processes input data using a sigmoid activation function. This activity also demonstrates how a multi-layer perceptron can learn patterns that are complex compared to those solved by a single layer perceptron.

Import Libraries

```
import numpy as np
```

Define the Sigmoid Function

```
def sigmoid(sum_func):
    return 1 / (1 + np.exp(-sum_func))
```

```
sigmoid(0)
```

```
0.5
```

```
np.exp(2)
```

```
7.38905609893065
```

```
np.exp(1)
```

```
2.718281828459045
```

```
sigmoid(40)
```

```
1.0
```

```
sigmoid(-20.5)
```

```
1.2501528648238605e-09
```

Input Layer to Hidden Layer

 Screenshot%202021-01-26%20at%2011.39.16%20am.png

Data Overview

This dataset consists of four observations representing every combination of two binary input variables. These observations are used as the training data for the multi-layer perceptron. The data is manually defined using a NumPy array and is suitable for demonstrating how a neural network learns binary classification problems. The characteristics of the dataset are as follows:

- Number of observations = 4
- Number of features = 2
- Data type = binary intergers (0 and 1)
- Missing values = none
- Target valuable = none

Define "Inputs, outputs and weights" as Numpy arrays

Inputs

Input exploration

The input dataset contains two binary features and contains all possible combinations of the two input variables. The dataset is complete, with no missing values, and is appropriate for training and testing the neural network.

Observation	Input 1	Input 2
1	0	0
2	0	1
3	1	0
4	1	1

```
inputs = np.array([[0,0],  
                  [0,1],  
                  [1,0],  
                  [1,1]])
```

inputs

```
array([[0, 0],  
       [0, 1],  
       [1, 0],  
       [1, 1]])
```

inputs.shape

```
(4, 2)
```

Outputs

Target variable

The target variable values demonstrate the expected output of the exclusive OR logical operation. The XOR function returns 0 when both inputs are the same and 1 when the inputs are different. Unlike the AND operator, the XOR dataset is not linearly separable, making it an appropriate example for demonstrating the capabilities of a multi-layer perceptron.

```
outputs = np.array([[0],  
                   [1],  
                   [1],  
                   [0]])
```

outputs.shape

```
(4, 1)
```

Output exploration

The target values are evenly distributed between the two classes, providing two examples of each output value.

Input 1	Input 2	Expected Output
0	0	0
0	1	1
1	0	1
1	1	0

Weights

Initial weights

The neural network requires weights to connect the neurons between layers. Two sets of weights are defined, weights_01 connects the input layer to the hidden layer, and weights_1 connects the hidden layer to the output layer. These weights are adjusted during training

to reduce prediction error.

- These weights are for the connection between the inputs and the hidden layer

```
# First row holds the weights for x1, 2nd row contains the weights for x2

weights_0 = np.array([[ -0.424, -0.740, -0.961],
                      [ 0.358, -0.577, -0.469]])

weights_0.shape

(2, 3)
```

- These weights are for the connection between the hidden layer and the output

```
weights_1 = np.array([[ -0.017],
                      [-0.893],
                      [ 0.148]])

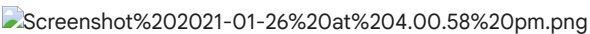
weights_1.shape

(3, 1)
```

- Epochs & Learning Rate

```
epochs = 100
learning_rate = 0.3
```

```
#for epoch in epochs:
```

Dealing with the first side 

```
input_layer = inputs
input_layer
```

```
array([[0, 0],
       [0, 1],
       [1, 0],
       [1, 1]])
```

```
# "sum_synapse_0" This holds the sum function total of weights for the hidden layer
# For the Output: Each row holds the sum_func for each input data [0,0,0 -> data 0,0],[0.358, -0.577, -0.469 --
# The dot product does the matrix multiplication and also the sum
```

```
sum_synapse_0 = np.dot(input_layer, weights_0)
sum_synapse_0
```

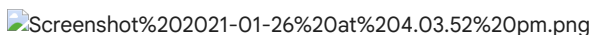
```
array([[ 0. ,  0. ,  0. ],
       [ 0.358, -0.577, -0.469],
       [-0.424, -0.74 , -0.961],
       [-0.066, -1.317, -1.43 ]])
```

```
# Computing the Sigmoid function for the Hidden layer
```

```
hidden_layer = sigmoid(sum_synapse_0)
hidden_layer
```

```
array([[0.5 , 0.5 , 0.5 ],
       [0.5885562, 0.35962319, 0.38485296],
       [0.39555998, 0.32300414, 0.27667802],
       [0.48350599, 0.21131785, 0.19309868]])
```

Dealing with the 2nd side



```
weights_1
```

```
array([[ -0.017],
       [-0.893],
       [ 0.148]])
```

```
# "sum_synapse_1" This holds the sum function total of weights for the output layer
# For the Output: Each row holds the sum_func for each input data
```

```
sum_synapse_1 = np.dot(hidden_layer, weights_1)
sum_synapse_1
```

```
array([[ -0.381      ],
       [ -0.27419072],
       [ -0.25421887],
       [ -0.16834784]])
```

```
output_layer = sigmoid(sum_synapse_1)
output_layer
```

```
array([[0.40588573],
       [0.43187857],
       [0.43678536],
       [0.45801216]])
```

 Screenshot%202021-01-26%20at%205.36.22%20pm.png

```
outputs
```

```
array([[0],
       [1],
       [1],
       [0]])
```

```
output_layer
```

```
array([[0.40588573],
       [0.43187857],
       [0.43678536],
       [0.45801216]])
```

```
error_output_layer = outputs - output_layer
error_output_layer
```

```
array([[ -0.40588573],
       [ 0.56812143],
       [ 0.56321464],
       [ -0.45801216]])
```

```
average_error = np.mean(abs(error_output_layer))
average_error
```

```
0.49880848923713045
```

Sigmoid Derivative  Screenshot%202021-01-27%20at%201.44.39%20am.png

```
def sigmoid_derivative(sigmoid):
    return sigmoid * (1 - sigmoid)
```

▼ Delta output Calculation

 Screenshot%202021-01-27%20at%2010.42.48%20am.png

```
# output_layer holds the results of our application of the sigmoid, computed above
```

```
output_layer
```

```
array([[0.40588573],
       [0.43187857],
       [0.43678536],
       [0.45801216]])
```

```
# derivative_output is our Derivative of the activation function (sigmoid) which we have on the slide
# each row is for each instance of our input dataset
```

```
derivative_output = sigmoid_derivative(output_layer)
derivative_output
```

```
array([[0.2411425 ],
       [0.24535947],
       [0.24600391],
       [0.24823702]])
```

```
error_output_layer
```

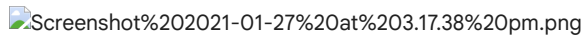
```
array([[ -0.40588573],
       [ 0.56812143],
       [ 0.56321464],
       [-0.45801216]])
```

```
# Delta output
# each row is for each instance of our input dataset

delta_output = error_output_layer * derivative_output
delta_output
```

```
array([[ -0.0978763 ],
       [ 0.13939397],
       [ 0.138553   ],
       [-0.11369557]])
```

Δ Delta calculations for the Hidden Layer

 Screenshot%202021-01-27%20at%203.17.38%20pm.png

```
delta_output
```

```
array([[ -0.0978763 ],
       [ 0.13939397],
       [ 0.138553   ],
       [-0.11369557]])
```

```
weights_1
```

```
array([[ -0.017],
       [-0.893],
       [ 0.148]])
```

Δ NOTE THAT:

- * Lets deal with this part first (Weight * delta_output)
- * Notice that we will get an error below because of the shape of the weights_1 (Transpose)

```
delta_output_x_weight = delta_output.dot(weights_1)
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-35-50b740e5a31c> in <module>
----> 1 delta_output_x_weight = delta_output.dot(weights_1)

ValueError: shapes (4,1) and (3,1) not aligned: 1 (dim 1) != 3 (dim 0)
```

```
weights_1.shape
```

```
(3, 1)
```

```
weights_1T = weights_1.T
weights_1T
```

```
array([[ -0.017, -0.893,  0.148]])
```

```
weights_1T.shape
```

```
(1, 3)
```

Δ Each one of the weights will have to be multiplied by each delta_output for each data instance

```
array([[ -0.017],
       [-0.893],
       [ 0.148]])
```

```
delta_output_x_weight = delta_output.dot(weights_1T)
delta_output_x_weight
```

```
array([[ 0.0016639 ,  0.08740354, -0.01448569],
       [-0.0023697 , -0.12447882,  0.02063031],
```

```
[-0.0023554 , -0.12372783,  0.02050584],
 [ 0.00193282,  0.10153015, -0.01682694]])
```

NOTE THAT:

* Now we need to deal with the last part of the equation
 * sigmoid_derivative * delta_output_x_weight

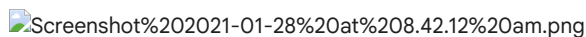
hidden_layer

```
array([[0.5      , 0.5      , 0.5      ],
       [0.5885562, 0.35962319, 0.38485296],
       [0.39555998, 0.32300414, 0.27667802],
       [0.48350599, 0.21131785, 0.19309868]])
```

Each row in the output of delta_hidden_layer is for the data input values

```
delta_hidden_layer = delta_output_x_weight * sigmoid_derivative(hidden_layer)
delta_hidden_layer
```

```
array([[ 0.00041597,  0.02185088, -0.00362142],
       [-0.00057384, -0.02866677,  0.00488404],
       [-0.00056316, -0.02705587,  0.00410378],
       [ 0.00048268,  0.01692128, -0.00262183]])
```

Screenshot%202021-01-28%20at%208.42.12%20am.png

We will deal with the (input * delta) first

- The first column in "hidden_layer" holds the activation value for the first neuron

hidden_layer

```
array([[0.5      , 0.5      , 0.5      ],
       [0.5885562, 0.35962319, 0.38485296],
       [0.39555998, 0.32300414, 0.27667802],
       [0.48350599, 0.21131785, 0.19309868]])
```

delta_output

```
array([[ -0.0978763 ],
       [ 0.13939397],
       [ 0.138553   ],
       [-0.11369557]])
```

- We need to multiply the "inputs" by "delta" however, for the matrix multiplication we need to transpose the values in the hidden_layer, so we have all of them on one row for each neuron

```
hidden_layerT = hidden_layer.T
hidden_layerT
```

```
array([[0.5      , 0.5885562 , 0.39555998, 0.48350599],
       [0.5      , 0.35962319, 0.32300414, 0.21131785],
       [0.5      , 0.38485296, 0.27667802, 0.19309868]])
```

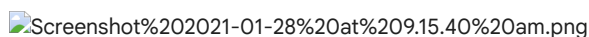
```
input_x_delta1 = hidden_layerT.dot(delta_output)
input_x_delta1
```

```
array([[0.03293657],
       [0.02191844],
       [0.02108814]])
```

Let us now update the "weights_1"

```
weights_1 = weights_1 + (input_x_delta1 * learning_rate)
weights_1
```

```
array([[ -0.00711903],
       [-0.88642447],
       [ 0.15432644]])
```

Screenshot%202021-01-28%20at%209.15.40%20am.png

Dealing with the Hidden Layer to Input Layer

 Screenshot%202021-01-28%20at%209.17.47%20am.png

```
# First column is X1, and 2nd column is X2 (our input values )
```

```
input_layer
```

```
array([[0, 0],
       [0, 1],
       [1, 0],
       [1, 1]])
```

```
delta_hidden_layer
```

```
array([[ 0.00041597,  0.02185088, -0.00362142],
       [-0.00057384, -0.02866677,  0.00488404],
       [-0.00056316, -0.02705587,  0.00410378],
       [ 0.00048268,  0.01692128, -0.00262183]])
```

```
# we need to transpose the values just as we did before
```

```
input_layerT = input_layer.T
```

```
input_layerT
```

```
array([[0, 0, 1, 1],
       [0, 1, 0, 1]])
```

```
input_x_delta0 = input_layerT.dot(delta_hidden_layer)
```

```
input_x_delta0
```

```
array([[ -8.04778516e-05, -1.01345901e-02,  1.48194623e-03],
       [-9.11603819e-05, -1.17454886e-02,  2.26221011e-03]])
```

```
weights_0 = weights_0 + (input_x_delta0 * learning_rate)
```

```
weights_0
```

```
array([[ -0.42402414, -0.74304038, -0.96055542],
       [ 0.35797265, -0.58052365, -0.46832134]])
```

 Screenshot%202021-01-28%20at%209.24.48%20am.png

So all the lines of code above, has allowed us to complete our first epoch. we will need to put all the code together so we can run multiple epochs

Complete Artificial Neural Network

```
#Importing Numpy
import numpy as np
```

```
# This is the sigmoid Function
```

```
def sigmoid(sum):
    return 1 / (1 + np.exp(-sum))
```

```
#This is the sigmoid derivative as used before
```

```
def sigmoid_derivative(sigmoid):
    return sigmoid * (1 - sigmoid)
```

```
# Our input values
```

```
inputs = np.array([[0,0],
                   [0,1],
                   [1,0],
                   [1,1]])
```

```
#Our output values
```

```
outputs = np.array([[0],
                    [1],
                    [1],
                    [0]])
```

```
# weights_0 = np.array([[ -0.424, -0.740, -0.961],
#                       [ 0.358, -0.577, -0.469]])
```

```
# weights_1 = np.array([[ -0.017],
```

```
# [-0.893],
# [0.148]])
```

▼ Initializing our weights with random values

- **Note:** Multiplying the random number by 2 and subtracting by 1, allows us to have a mix of both positive and negative random numbers for the weights

```
weights_0 = 2 * np.random.random((2, 3)) - 1
weights_1 = 2 * np.random.random((3, 1)) - 1
```

```
epochs = 400000
learning_rate = 0.6
error = []

for epoch in range(epochs):
    input_layer = inputs
    sum_synapse0 = np.dot(input_layer, weights_0)
    hidden_layer = sigmoid(sum_synapse0)

    sum_synapse1 = np.dot(hidden_layer, weights_1)
    output_layer = sigmoid(sum_synapse1)

    error_output_layer = outputs - output_layer
    average = np.mean(abs(error_output_layer))
    #print after every specified range of the value
    if epoch % 100000 == 0:
        print('Epoch: ' + str(epoch + 1) + ' Error: ' + str(average))
        error.append(average)

    derivative_output = sigmoid_derivative(output_layer)
    delta_output = error_output_layer * derivative_output

    weights1T = weights_1.T
    delta_output_weight = delta_output.dot(weights1T)
    delta_hidden_layer = delta_output_weight * sigmoid_derivative(hidden_layer)

    hidden_layerT = hidden_layer.T
    input_x_delta1 = hidden_layerT.dot(delta_hidden_layer)
    weights_1 = weights_1 + (input_x_delta1 * learning_rate)

    input_layerT = input_layer.T
    input_x_delta0 = input_layerT.dot(delta_hidden_layer)
    weights_0 = weights_0 + (input_x_delta0 * learning_rate)
```

```
Epoch: 1 Error: 0.4999269690201742
Epoch: 100001 Error: 0.015563199459768319
Epoch: 200001 Error: 0.010326071808337757
Epoch: 300001 Error: 0.008192022809586367
```

▼ At this point after running for 1million epochs you can see the value is very low.

```
#1 million epochs with a learning rate of 0.3
1 - 0.009670967930930745
```

```
0.9903290320690693
```

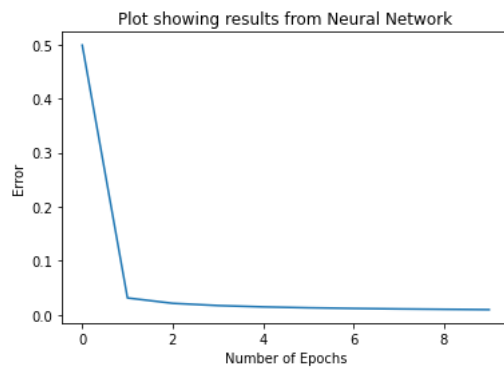
```
#after 400,000 epochs, with a learning rate of 0.6
1- 0.008192022809586367
```

```
0.9918079771904136
```

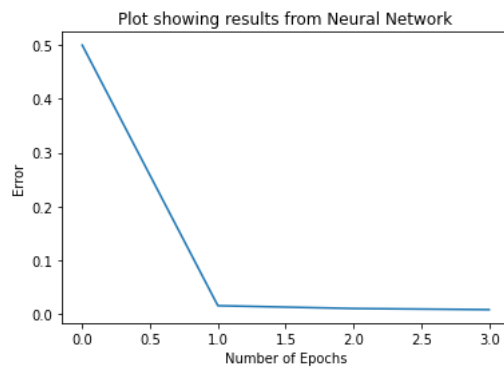
▼ Let's visualize this result

```
import matplotlib.pyplot as plt
```

```
plt.xlabel('Number of Epochs')
plt.ylabel('Error')
plt.title('Plot showing results from Neural Network')
plt.plot(error)
plt.show()
```



```
plt.xlabel('Number of Epochs')
plt.ylabel('Error')
plt.title('Plot showing results from Neural Network')
plt.plot(error)
plt.show()
```



✓ Comparing the outputs and the predictions

outputs

```
array([[0],
       [1],
       [1],
       [0]])
```

output_layer

```
array([[0.0105802 ],
       [0.9826343 ],
       [0.98124727],
       [0.02222762]])
```

✓ * We see that our neural network was able to get values close to the actual values from the results.

* This shows that our neural network can handle the complexity of the XOR operator dataset.

- Let us see the updated weights. These are the weights we will require if we want to make future predictions

weights_0

```
array([[ -0.04578885, -5.45125419, -1.01868082],
       [ 1.04772434, -5.25578328, -0.5958262 ]])
```

weights_1

```
array([[ -14.93268248],
       [-37.16040415],
       [ 43.01681387]])
```

This function accepts an instance of a dataset

```
def calculate_output(instance):
    #input to hidden layer
    hidden_layer = sigmoid(np.dot(instance, weights_0))
    #hidden to output layer
```

```
output_layer = sigmoid(np.dot(hidden_layer, weights_1))  
return output_layer[0]
```

```
round(calculate_output(np.array([0, 0])))
```

```
0
```

```
round(calculate_output(np.array([0, 1])))
```

```
1
```

```
round(calculate_output(np.array([1, 0])))
```

```
1
```

```
round(calculate_output(np.array([1, 1])))
```

```
0
```

✓ Conclusion

This activity demonstrated how a multi-layer perceptron uses the sigmoid activation function to learn non-linear classification problem. The XOR dataset contains four binary observations and cannot be correctly classified by a single-layer perceptron. By training the network and adjusting the weights, the multi-layer perceptron successfully learned the relationship between the input features and the expected outputs. This exercise highlights how hidden layers and the sigmoid activation function enable neural networks to solve more complex classification problems than a simple perceptron.

Start coding or [generate](#) with AI.